

第4章 快速傅里叶变换 (FFT)

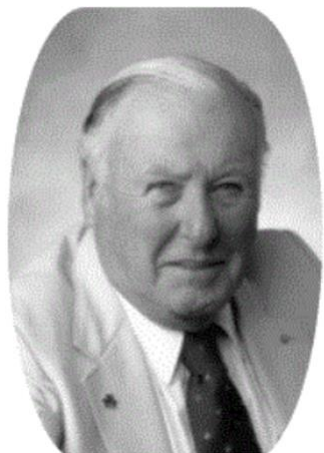
本章主要内容

- DFT高效计算的思路
- **基2FFT算法：** DIT-基2FFT、DIF-基2FFT、IDFT快速计算方法

- DFT是信号分析与处理中的一种重要变换。但直接计算DFT的**计算量大**，实时对信号进行谱分析和实时处理的成本较高。
- 1965年图基和库利提出了DFT的快速算法，使DFT的运算效率提高**1~2个数量级**，使得DFT得到了广泛应用，推动了数字处理技术的发展。
- 1984年，提出了**分裂基**快速算法，使运算效率进一步提高；



图基 (John Tukey 1915 -2000)，美国著名统计学家，1939年在普林斯顿大学教书，统计学系系主任，还在贝尔实验室工作了几十年。在倒谱方面的工作对**声码器**的发展和其他领域产生影响。创造了“**比特 (Bit)**” “**软件 (Software)**” 两个词。



库利 (James W. Cooley 1926 -2016)，美国著名数学家，1949年在曼哈顿学院获得学士学位，1951年获得数学硕士学位，在哥伦比亚大学获得应用数学博士学位。

1965年**Tukey** 和**Cooley** 发表在《**Mathematics of Computation**》杂志上的一篇文章《**An Algorithm for the Machine Calculation of Complex Fourier Series**》，提出了FFT算法。**FFT算法**彻底改变了**傅里叶的应用**，极大地改变了频域和时域解决问题的方法，**推动数字信号处理快速发展**，对现在的**网络通信**，**图形图像处理**等领域发展奠定了基础。

4.1 DFT高效计算的思路

一、直接计算DFT的特点及减少运算量的途径

1、直接计算DFT

长度为N的有限长序列 $x(n)$ ，其N点DFT为：

$$X(k) = \sum_{n=0}^{N-1} x(n)W_N^{kn}, k = 0, 1, \dots, N-1$$

考虑 $x(n)$ 为复序列的一般情况，对某一个k值，直接计算 $X(k)$ 值需要N次复数乘法、(N-1)次复数加法。

N点DFT的复乘次数等于 N^2 ，复加次数为 $N(N-1)$ （不考虑特殊值）

2、减少运算量的思路

减小N？把N点长序列的DFT分解为几个较短序列的DFT，可使乘法次数减少？

3、旋转因子的特点

旋转因子 W_N^k 具有周期性、对称性和可约性。

周期性: $W_N^{kn} = W_N^{k(n+LN)} = W_N^{(k+LN)n}$

周期为N

对称性: $(W_N^{kn})^* = W_N^{-kn}$

可约性: $W_{mN}^{kmn} = W_N^{kn} \quad W_N^{kn} = W_{N/m}^{kn/m}$

特殊值: $W_N^{N/2} = -1, \quad W_N^{k+N/2} = -W_N^k$

4、FFT算法思想

不断地把一个长序列的DFT分解成几个短序列的DFT，并利用旋转因子的周期性、对称性和可约性来减少DFT的运算次数。

FFT算法分为两大类：时域抽取法FFT(简称DIT-FFT)

频域抽取法FFT(简称DIF-FFT)

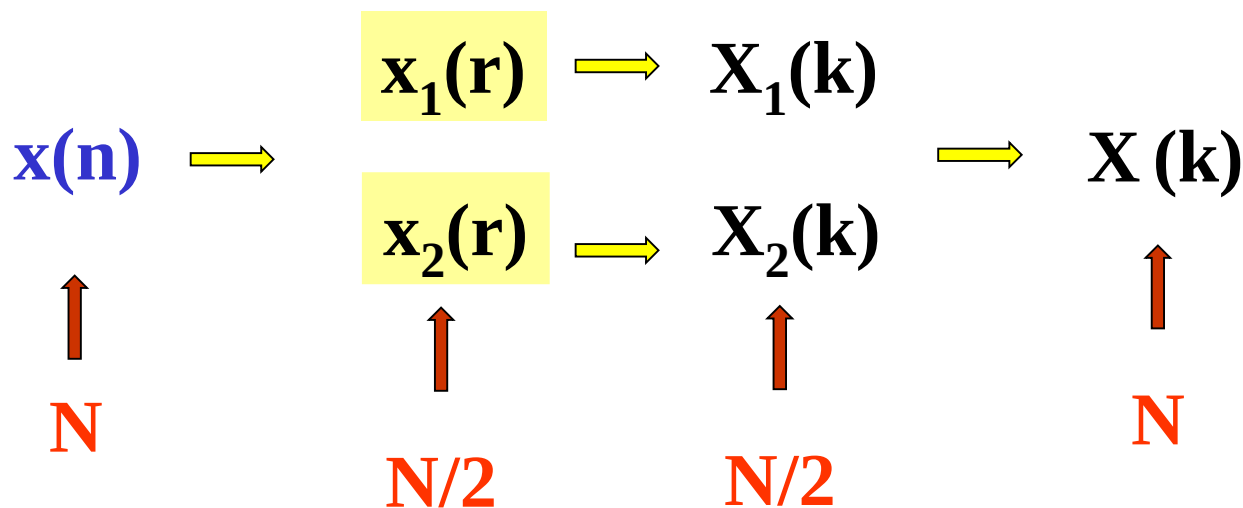


4.2 时域抽取 (DIT) 基-2FFT基本原理

一、算法原理

设序列 $x(n)$ 的长度为 N ，且满足： $N = 2^M$ ， M 为自然数

- (1) 将序列 $x(n)$ 按 n 为奇、偶数分为 $x_1(n)$ 和 $x_2(n)$ 两组序列；
- (2) 用2个 $N/2$ 点的DFT来完成一个 N 点DFT的计算。



(1) 按n为奇数还是偶数，把x(n)分解为两个N/2点的子序列

$$\begin{cases} x_1(r) = x(2r) \\ x_2(r) = x(2r+1) \end{cases} \quad r = 0, 1, \dots, \frac{N}{2} - 1$$

(2) 用N/2点 $X_1(k)$ 和 $X_2(k)$ 表示序列x(n)的N点DFT $X(k)$

$$X(k) = \sum_{n=\text{偶数}} x(n)W_N^{kn} + \sum_{n=\text{奇数}} x(n)W_N^{kn} = \sum_{r=0}^{N/2-1} x(2r)W_N^{2kr} + \sum_{r=0}^{N/2-1} x(2r+1)W_N^{k(2r+1)}$$

$$= \sum_{r=0}^{N/2-1} x_1(r)W_N^{2kr} + W_N^k \sum_{r=0}^{N/2-1} x_2(r)W_N^{2kr}$$

$$W_N^{2kr} = W_{N/2}^{kr}$$

$$= \sum_{r=0}^{N/2-1} x_1(r)W_{N/2}^{kr} + W_N^k \sum_{r=0}^{N/2-1} x_2(r)W_{N/2}^{kr} = X_1(k) + W_N^k X_2(k)$$

注意：这里的k的取值范围为0, 1, ..., N-1

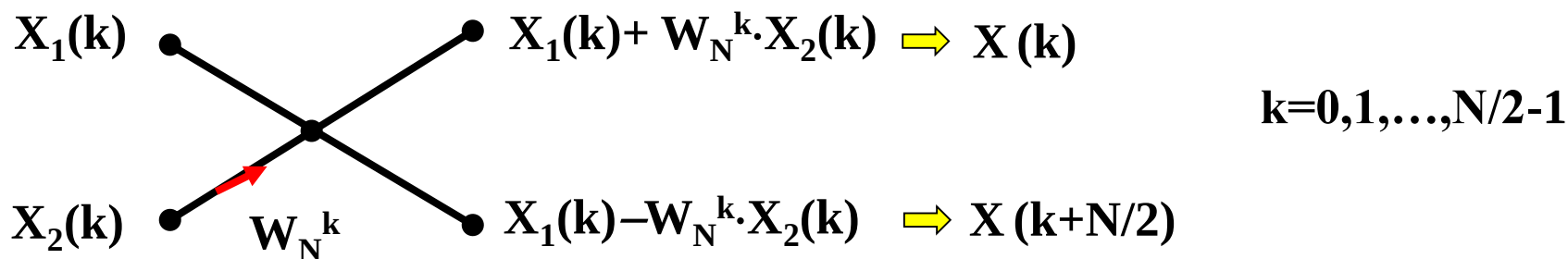
$$X(k) = X_1(k) + W_N^k X_2(k) \quad k=0,1,\dots,N-1$$

由于 $X_1(k)$ 和 $X_2(k)$ 均以 $N/2$ 为周期，且 $W_N^{k+\frac{N}{2}} = -W_N^k$ ， $X(k)$ 可表示为：

$$\begin{cases} X(k) = X_1(k) + W_N^k X_2(k) \\ X(k + \frac{N}{2}) = X_1(k) - W_N^k X_2(k) \end{cases} \quad k = 0, 1, \dots, \frac{N}{2} - 1$$

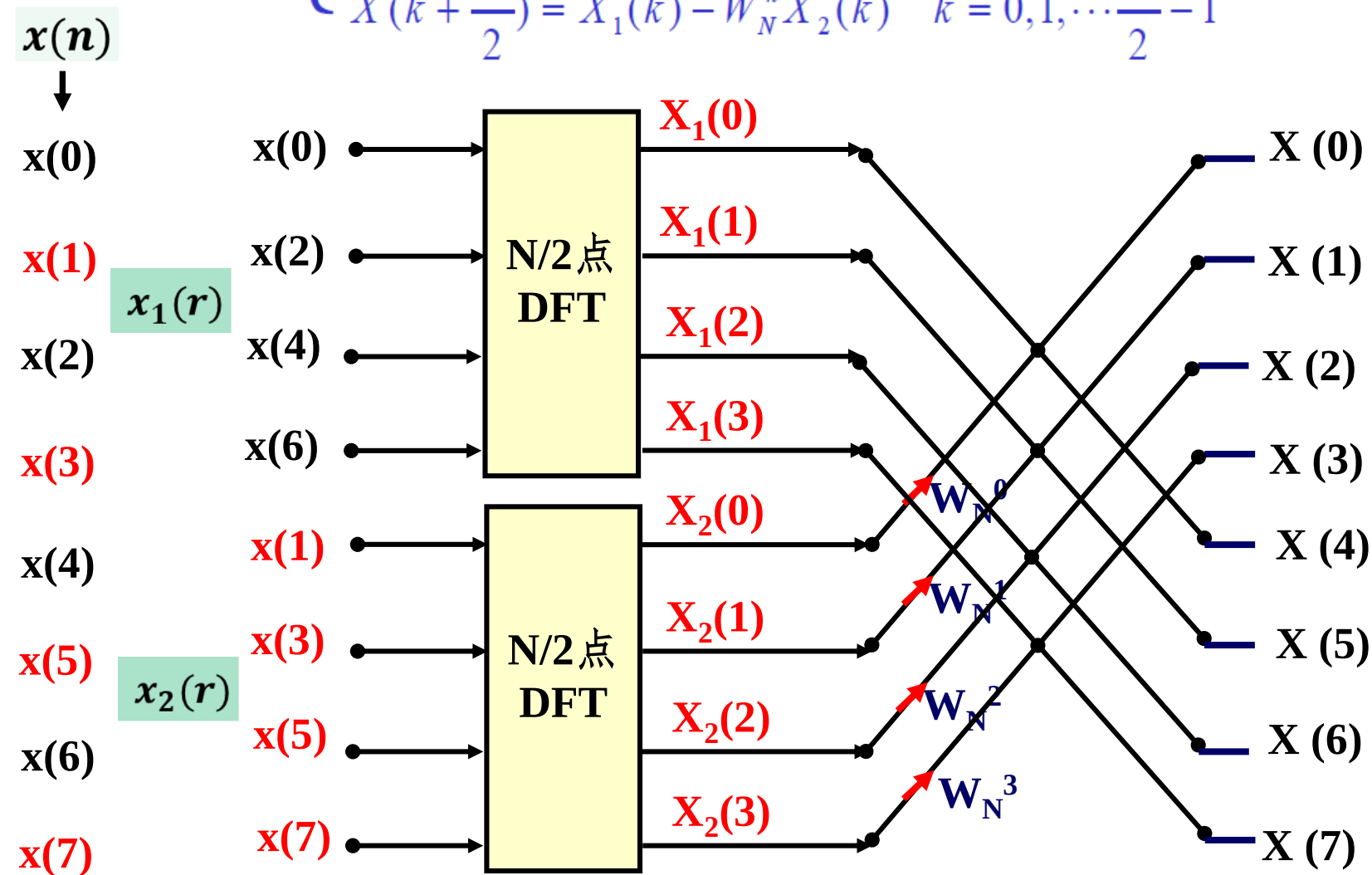
将N点DFT分解为
两个N/2点的DFT

对上式的运算可用N个蝶形运算流图符号来表示



- 完成一个蝶形运算需要一次复数乘和两次复数加法运算；
- 经过一次分解后，共需要复数乘和复数加的次数为 $(N+1)N/2$ 和 $N^2/2$

$$\begin{cases} X(k) = X_1(k) + W_N^k X_2(k) & k = 0, 1, \dots, \frac{N}{2} - 1 \\ X(k + \frac{N}{2}) = X_1(k) - W_N^k X_2(k) & k = 0, 1, \dots, \frac{N}{2} - 1 \end{cases}$$



N=8点的DIT-2FFT一次时域抽取分解图

(3) 第二次分解

- 将 $x_1(r)$ 按 r 取奇、偶可分解成2个长度为 $N/4$ 的子序列

$$\begin{cases} x_3(l) = x_1(2l) \\ x_4(l) = x_1(2l+1) \end{cases} \quad l = 0, 1, \dots, N/4 - 1$$

根据上面推导可得： $X_1(k) = X_3(k) + W_{N/2}^k X_4(k)$ $k=0, 1, \dots, N/2-1$

$$\begin{cases} X_1(k) = X_3(k) + W_{N/2}^k X_4(k) \\ X_1(k + N/4) = X_3(k) - W_{N/2}^k X_4(k) \end{cases} \quad k=0, 1, \dots, N/4-1$$

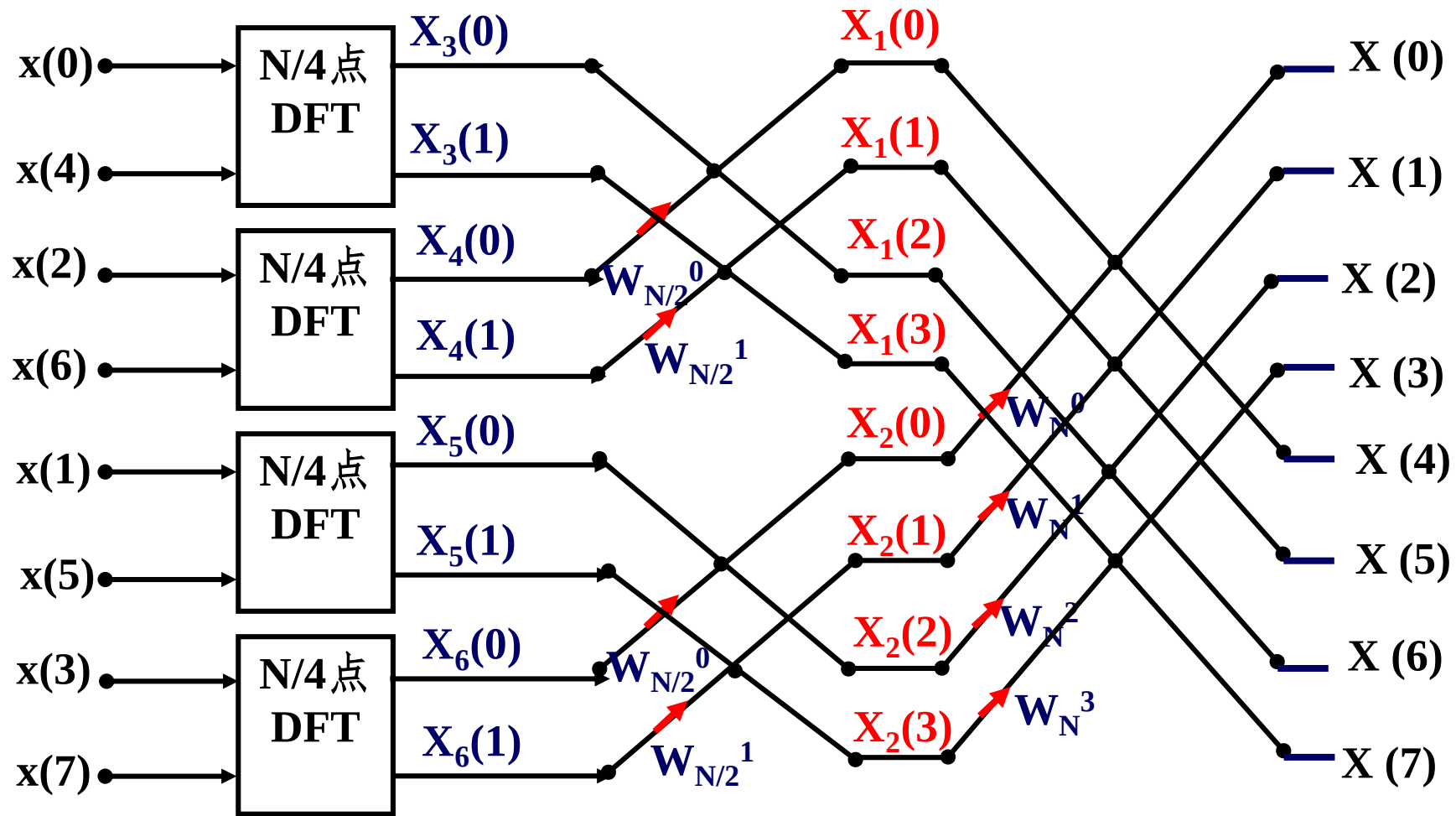
- 将 $x_2(r)$ 按 r 取奇、偶可分解成2个长度为 $N/4$ 的子序列

$$\begin{cases} x_5(l) = x_2(2l) \\ x_6(l) = x_2(2l+1) \end{cases} \quad l = 0, 1, \dots, N/4 - 1$$

根据上面推导可得： $X_2(k) = X_5(k) + W_{N/2}^k X_6(k) \quad k=0, 1, \dots, N/2-1$

$$\begin{cases} X_2(k) = X_5(k) + W_{N/2}^k X_6(k) \\ X_2(k + N/4) = X_5(k) - W_{N/2}^k X_6(k) \end{cases} \quad k=0, 1, \dots, N/4-1$$

N=8点DFT的二次时域抽取分解图



$$X_1(k) = X_3(k) + W_{N/2}^k X_4(k)$$

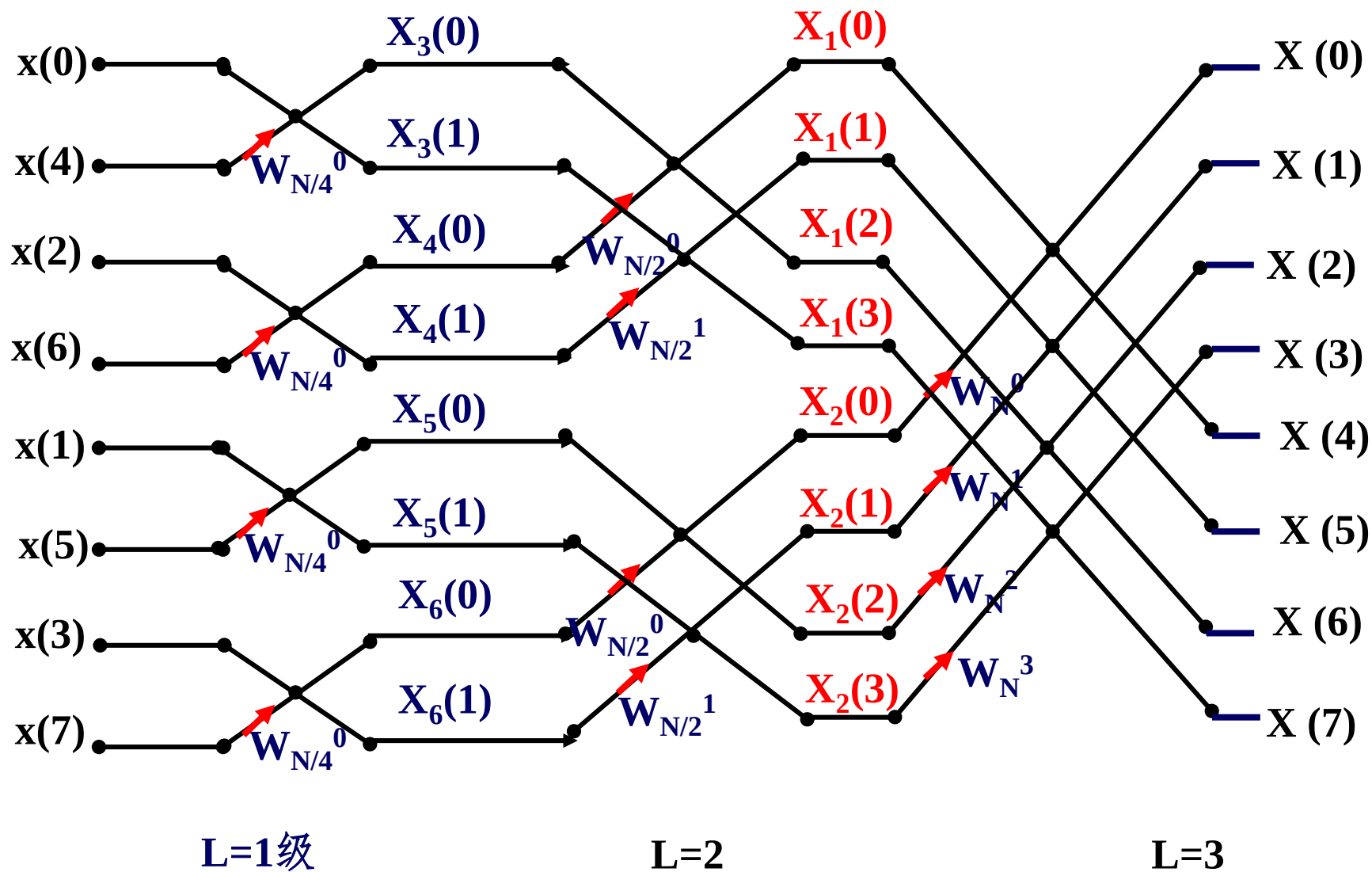
$$X_1(k + N/4) = X_3(k) - W_{N/2}^k X_4(k)$$

$$X_2(k) = X_5(k) + W_{N/2}^k X_6(k)$$

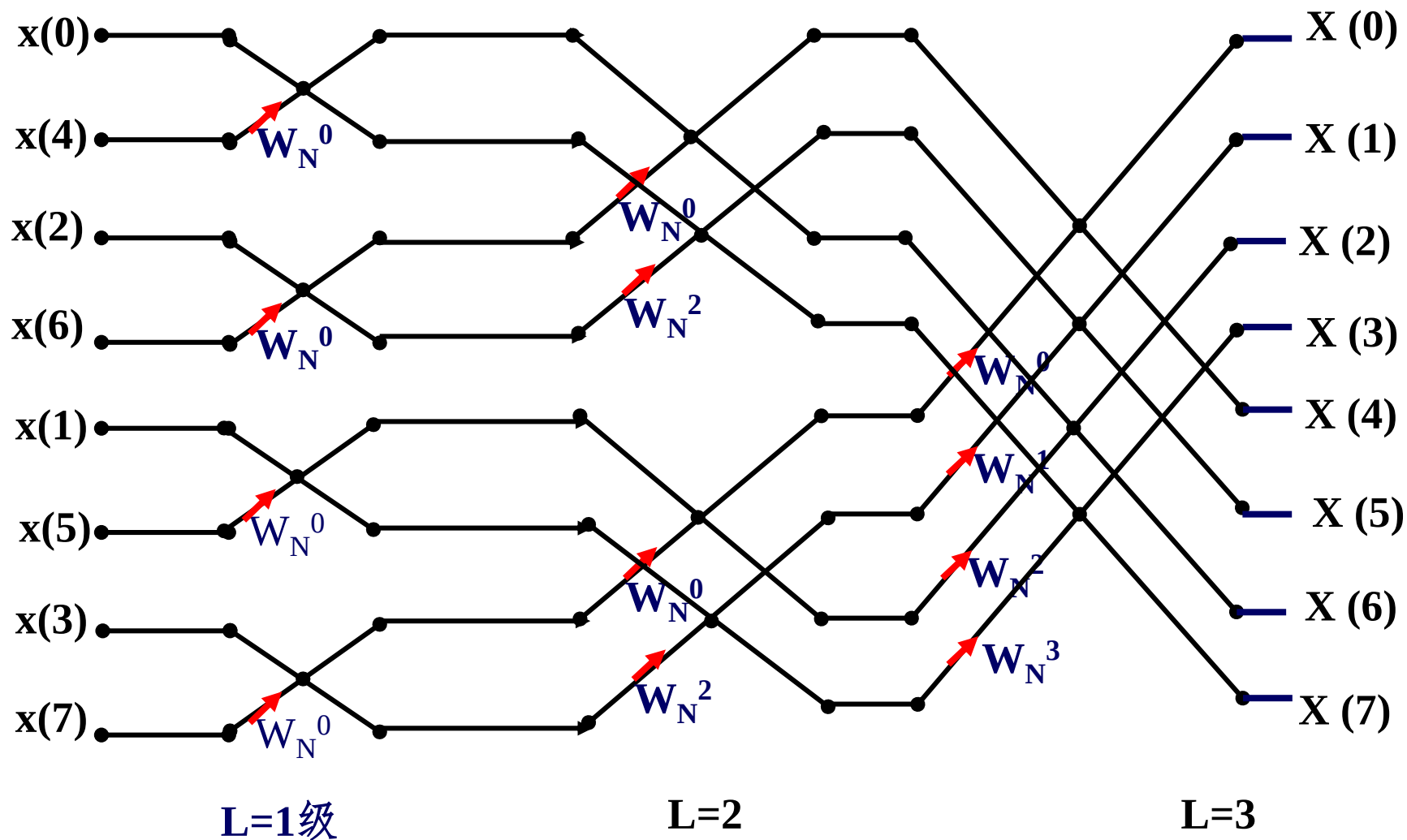
$$X_2(k + N/4) = X_5(k) - W_{N/2}^k X_6(k)$$

$$k=0,1,\dots,N/4-1;$$

N=8点DFT的三次时域抽取分解图



N=8点DIT-FFT运算流图



二、DIT-FFT算法的运算量分析

1、直接DFT运算N点运算

复数乘次数： $N \times N$

复数加次数： $N \times (N - 1)$

2、用DIT-FFT作N点运算

复数乘次数： $M \times \frac{N}{2} = \frac{N}{2} \times \log_2 N$

复数加次数： $M \times \frac{N}{2} \times 2 = N \times \log_2 N$

运算流图中有**M级**蝶形，每一级运算流图中有**N/2个**蝶形，每个蝶形需**一次复数乘**和**两次复数加**运算。

可见采用FFT能大大减少DFT的运算量，提高运算速度。

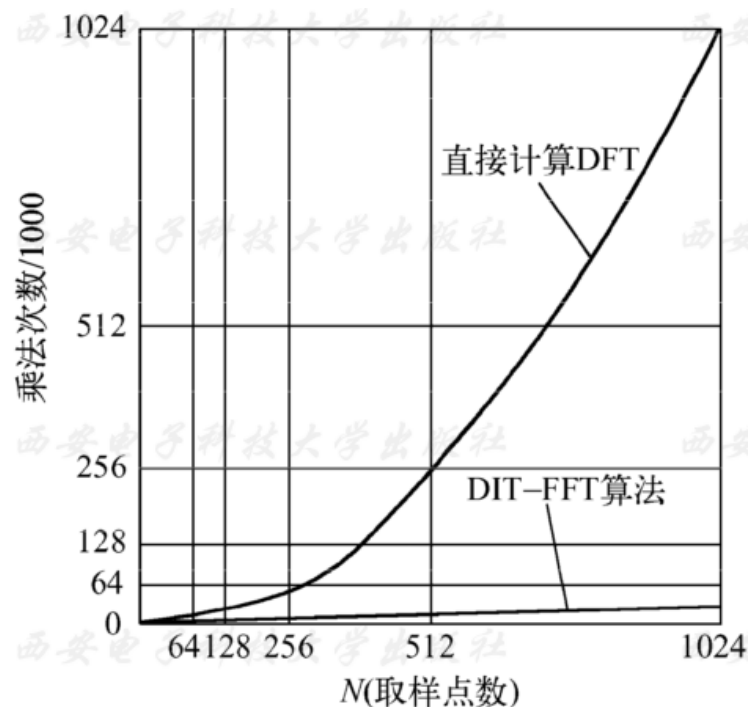
例1：某复序列 $x(n)$ 长度 $N=1024$ ，比较直接计算DFT的复数乘次数和采用DIT-FFT算法计算的复数乘次数。

解：直接计算DFT： $N^2=1024^2=1048576$

采用DIT-FFT算法计算： $N/2 \times \log N = 512 \times 10 = 5120$

运算效率 $1048576 / 5120 = 204.8$

N 越大，运算效率越高，FFT的优越性越明显



复数乘法次数的比较曲线

三、DIT-FFT算法的运算特点及编程思想

特点1：原位计算

序列长为 $N=2^M$ 点的FFT，有 M 级蝶形，每级有 $N/2$ 个蝶形运算。

同一级中，每个蝶形的两个输入数据只对本蝶形有用，每个蝶形的输入、输出数据节点在用一条水平线上。这样，当计算完一个蝶形后，所得的输出数据可立即存入原输入数据所占用的存储单元。经过 M 级运算后，原来存放输入序列数据 $x(n)$ 的 N 个存储单元中可依次存放 $X(k)$ 的 N 个值。

- 原位计算：利用同一存储单元存储蝶形输入、输出数据的方法。
- 优点：节约存储空间、降低设备的实现成本。

特点2：旋转因子的变化规律（指数P与级数L的关系）

■ 每个蝶形都要乘以旋转因子 W_N^p ， p 称为旋转因子的指数。 $N=8=2^3$ 时各级的旋转因子为：

● 第一级： $L=1$ ，有1个旋转因子： $W_{N/4}^J = W_{2^L}^J \quad J=0$

● 第二级： $L=2$ ，有2个旋转因子： $W_{N/2}^J = W_{2^L}^J \quad J=0, 1$

● 第三级： $L=3$ ，有4个旋转因子： $W_N^J = W_{2^L}^J \quad J=0, 1, 2, 3$

说明：每一级有 2^{L-1} 个旋转因子

对于 $N=2^M$ 的一般情况，第 L 级共有 2^{L-1} 个不同的旋转因子：

$$W_{2^L}^J = W_{2^M \times 2^{-M} \times 2^L}^J = W_{N \times 2^{L-M}}^J = W_N^{J \times 2^{M-L}} = W_N^P \quad \Rightarrow \quad \begin{aligned} P &= J \times 2^{M-L} \\ J &= 0, 1, \dots, 2^{L-1} - 1 \end{aligned}$$

结论：只要知道级数 L ，就可确定旋转因子的指数 p 和个数 J 。

特点3：同一级中，同一旋转因子对应蝶形运算的个数

第L级FFT运算流图中，同一旋转因子用在 2^{M-L} 个蝶形中。

特点4：同一级中，使用相同旋转因子的蝶形之间相隔的“距离”

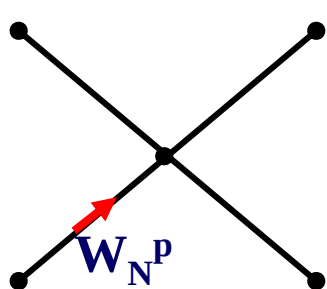
第L级中，蝶距： $D=2^L$ 。

特点5：同一蝶形运算两输入数据的距离

第L级中，每个蝶形的两输入数据相隔距离： $B=2^{L-1}$ 。

特点6：蝶形运算

蝶形的两个输入数据相距B个点，采用原位计算，蝶形的计算如下：

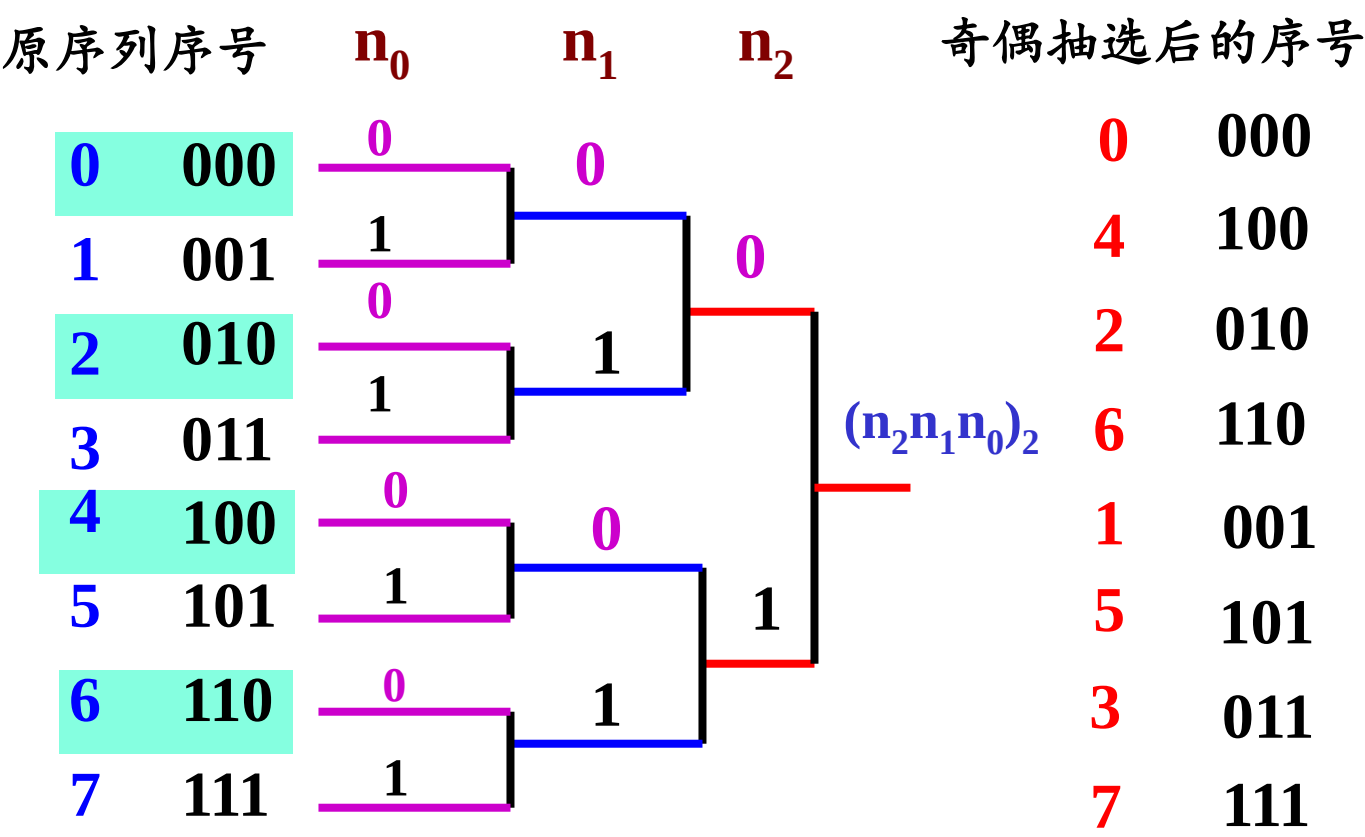

$$\begin{aligned} A_{L-1}(J) & \quad A_L(J) = A_{L-1}(J) + W_N^p A_{L-1}(J+B) \\ A_{L-1}(J+B) & \quad A_L(J+B) = A_{L-1}(J) - W_N^p A_{L-1}(J+B) \end{aligned}$$

$$B=2^{L-1}$$

$$p=J \times 2^{M-L}, \quad J=0, 1, 2, \dots, 2^{L-1}-1$$

特点7：倒序（码位颠倒）

$x(n)$ 经M次时域的奇偶抽选后，序列的序号与原序列序号之间的关系为倒序关系。
 $N=2^M$ ，用M位二进制数 $(n_{M-1}n_{M-2}\cdots n_1n_0)_2$ 表示序列的序号n。M次时域奇偶抽选过程为：最低位按0、1分为偶、奇两组；次低位按0、1分组，依此类推。



可见，只要将顺序数的二进制位倒置可得到对应的二进制倒序值。

二进制数(N=8)分解倒序图

例2：已知N=16点长的序列，采用DIT-基2FFT做16点DFT。在原序列中，序号为2，经倒序后的序号变为多少？
 (0010→0100) 倒序后，序号变为4

顺序和倒序二进制数对照表

顺序数是在二进制数的最低位加1，向左进位。

顺 序		倒 序	
十进制数 I	二 进 制 数	二 进 制 数	十进制数 J
0	0 0 0	0 0 0	0
1	0 0 1	1 0 0	4
2	0 1 0	0 1 0	2
3	0 1 1	1 1 0	6
4	1 0 0	0 0 1	1
5	1 0 1	1 0 1	5
6	1 1 0	0 1 1	3
7	1 1 1	1 1 1	7

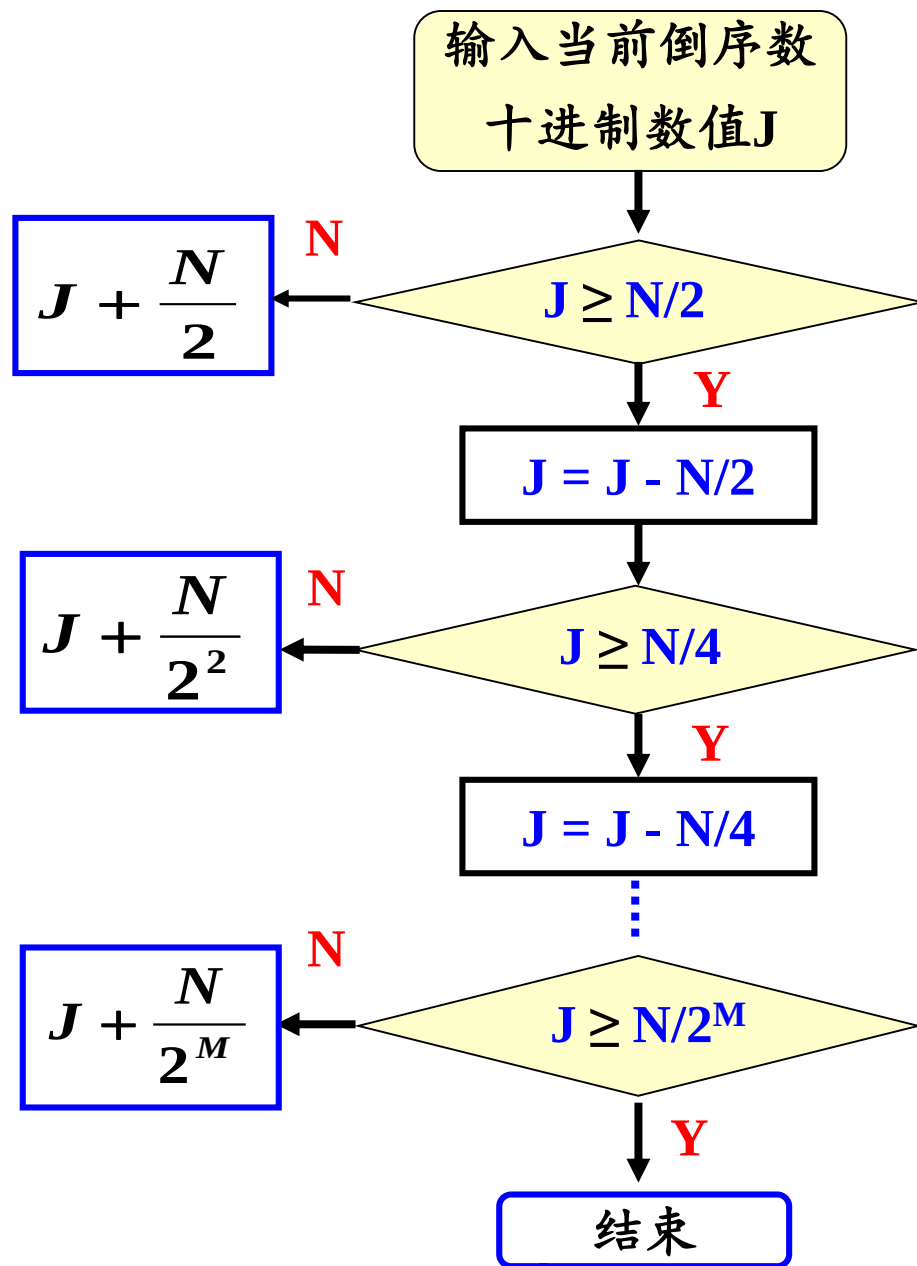
倒序数是在M位二进制数的最高位加1（加权值），向右进位。

N/2、N/4、N/8

权值

倒序数的十进制运算规律

- $N=2^M$ ，序号用M位二进制数表示，则从左至→右的十进制权值为： $N/2, N/4, N/8, \dots, 2, 1$
- 对倒序数J，其下一个序号数是在该序号数J的二进制首位码加1，相当于十进制运算 $J+N/2$ 。
- 计算机上倒序数的实现过程见流程图：



倒序的实现

输入序列

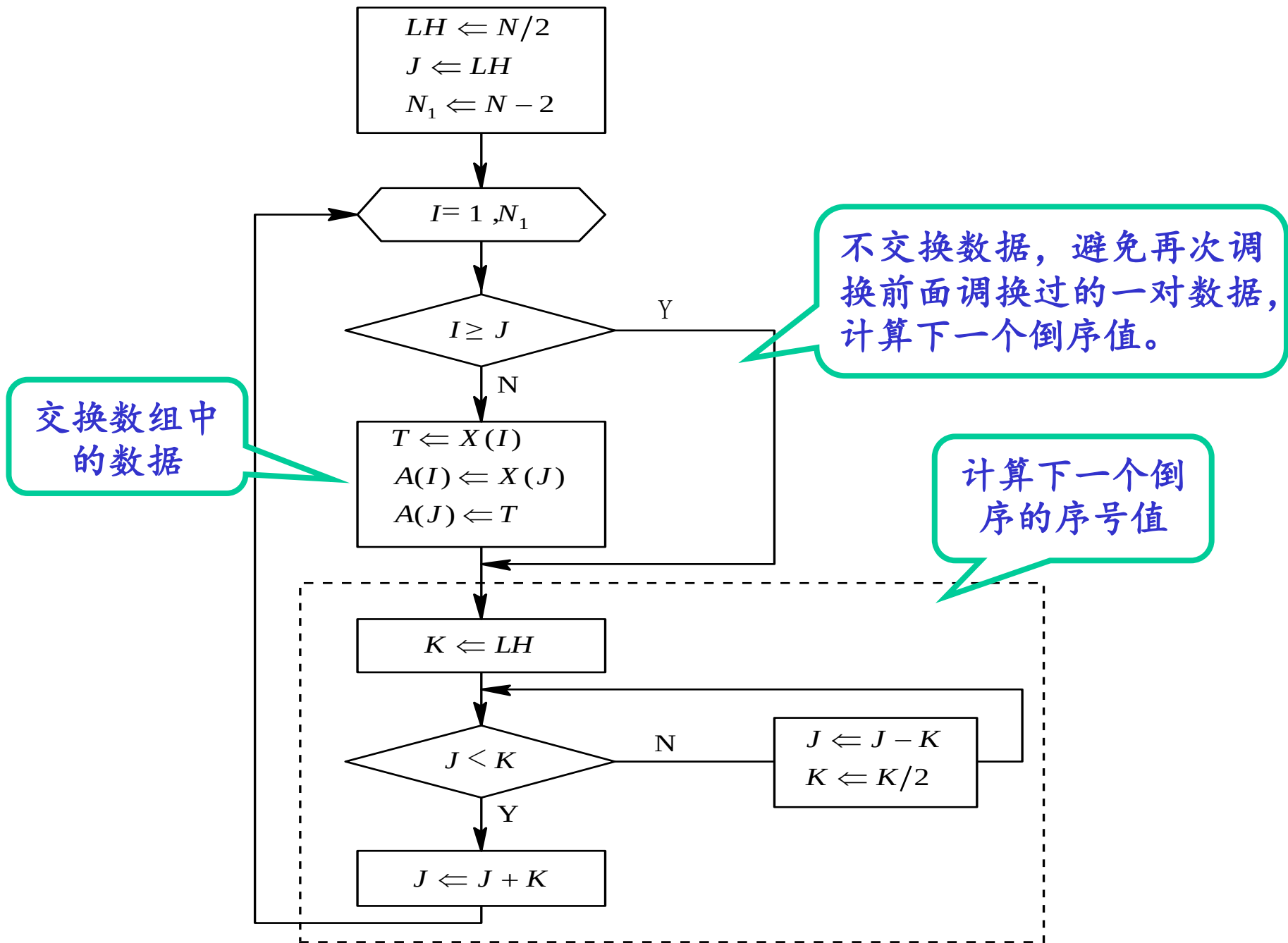
x(0) x(1) x(2) x(3) x(4) x(5) x(6) x(7)

倒序前的序号

倒序后的序号

I	J		
↓	↓		
0	0	A[0]	x(0)
1	4	A[1]	x(1)
2	2	A[2]	x(2)
3	6	A[3]	x(3)
4	1	A[4]	x(4)
5	5	A[5]	x(5)
6	3	A[6]	x(6)
7	7	A[7]	x(7)

- 结论：
- I < J时，交换存储器内容。
 - I ≥ J时，不交换，直接更新序号；



DIT-基2FFT程序实现框图

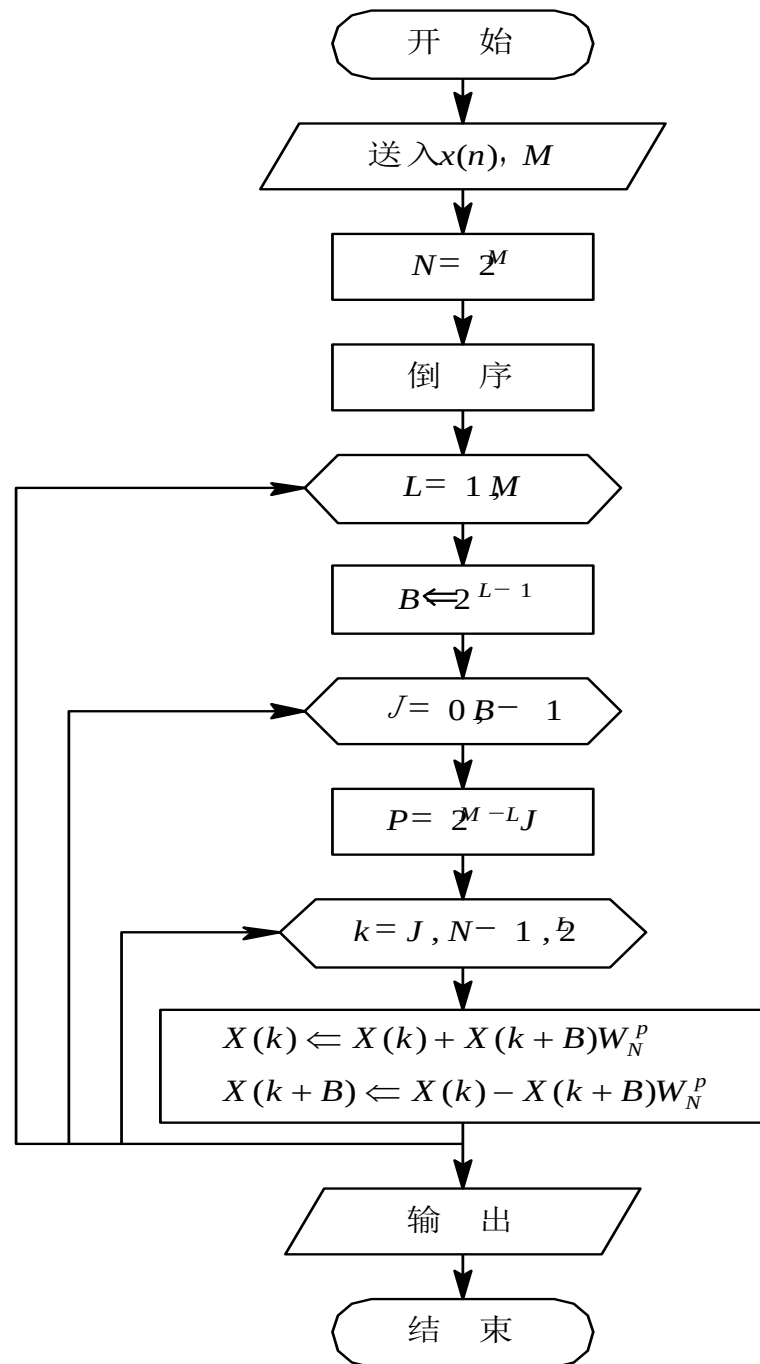
(1) 倒序：根据倒序规律，对输入自然顺序序列 $x(n)$ 进行倒序处理；

(2) 循环层1：确定运算的级数， $L=1\sim M$ ($N=2^M$)；确定一蝶形两输入数据距离 $B=2^{L-1}$

(3) 循环层2：确定 L 级的 2^{L-1} (B) 个旋转因子，计算旋转因子指数 $p=2^{M-L}J$ ， $J=0\sim B-1$ ；

(3) 循环层3：同一旋转因子作用于同一级的 2^{M-L} 个蝶形运算中， k 的取值从 J 到 $N-1$ ，步长为 2^L (使用同一旋转因子的蝶形相距的距离)

(5) 完成一个蝶形运算



4.3 频域抽取(DIF)法-基2FFT

1、算法思想和运算过程

设序列 $x(n)$ 长度为 $N=2^M$ ，将序列 $x(n)$ 前后对半分为 $x_1(n)$ 、 $x_2(n)$ 两组序列，用2个 $N/2$ 点的DFT来完成一个 N 点DFT的计算。

$$X(k) = DFT[x(n)] = \sum_{n=0}^{N-1} x(n)W_N^{kn} = \sum_{n=0}^{N/2-1} x(n)W_N^{kn} + \sum_{n=N/2}^{N-1} x(n)W_N^{kn}$$

$$= \sum_{n=0}^{N/2-1} x(n)W_N^{kn} + \sum_{n=0}^{N/2-1} x(n + \frac{N}{2})W_N^{k(n+N/2)}$$

$$= \sum_{n=0}^{N/2-1} [x(n) + W_N^{kN/2} x(n + \frac{N}{2})]W_N^{kn}$$

$$W_N^{kN/2} = (-1)^k = \begin{cases} 1, & k=\text{偶数} \\ -1, & k=\text{奇数} \end{cases}$$

将 $X(k)$ 分解成偶数组与奇数组，当 k 取偶数($k=2r$, $r=0,1,\dots,N/2-1$)时

$$X(2r) = \sum_{n=0}^{N/2-1} [x(n) + x(n + \frac{N}{2})] W_N^{2rn} = \sum_{n=0}^{N/2-1} \underbrace{[x(n) + x(n + N/2)]}_{x_1(n)} W_{N/2}^{rn}$$

当 k 取奇数($k=2r+1$, $r=0,1,\dots,N/2-1$)时：

$$X(2r+1) = \sum_{n=0}^{N/2-1} [x(n) - x(n + \frac{N}{2})] W_N^{n(2r+1)} = \sum_{n=0}^{N/2-1} \underbrace{[x(n) - x(n + \frac{N}{2})]}_{x_2(n)} W_N^n W_{N/2}^{nr}$$

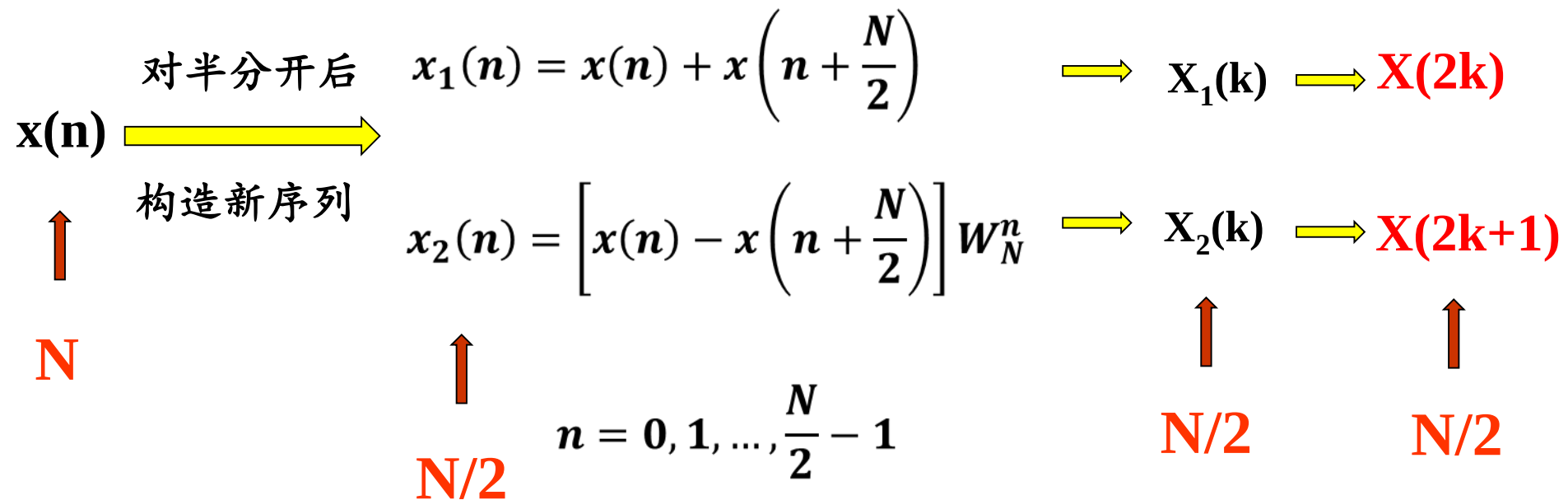
将 $x_1(n)$ 和 $x_2(n)$ 分别代入上式，可得

$$\begin{cases} X(2r) = \sum_{n=0}^{N/2-1} x_1(n) W_{N/2}^{rn} \\ X(2r+1) = \sum_{n=0}^{N/2-1} x_2(n) W_{N/2}^{nr} \end{cases} \quad r=0,1,\dots,N/2-1$$

表明：

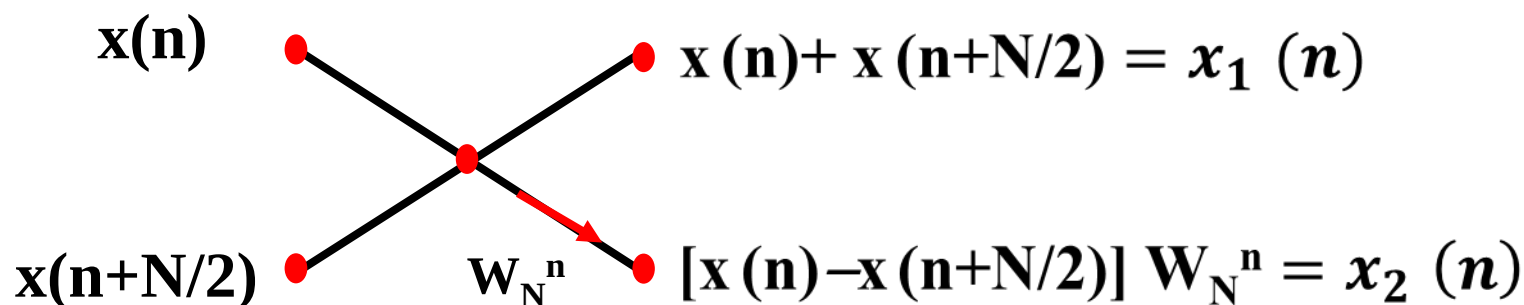
- $X(k)$ 按奇偶 k 值分为两组；
- 偶数组是 $x_1(n)$ 的 $N/2$ 点DFT；
- 奇数组是 $x_2(n)$ 的 $N/2$ 点DFT

频域抽取 (DIF) 基-2FFT分解过程的再理解



$$\begin{cases} x_1(n) = x(n) + x(n + N/2) \\ x_2(n) = [x(n) - x(n + N/2)] W_N^n \end{cases} \quad n=0, 1, \dots, N/2-1$$

对序列 $x_1(n)$ 、 $x_2(n)$ 和 $x(n)$ 之间的运算关系可用蝶形运算符号表示



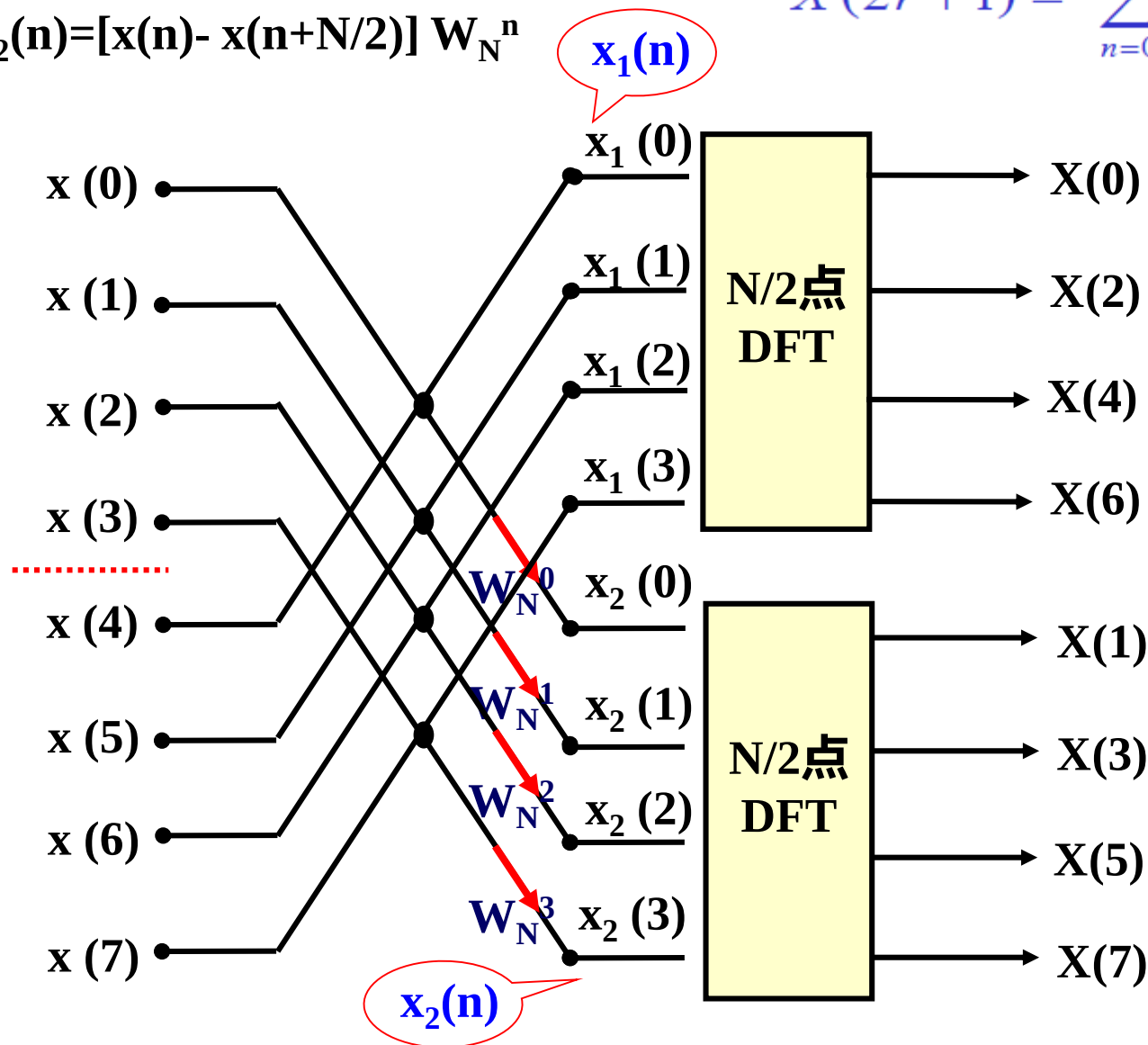
蝶形运算流图符号

- 完成一个蝶形运算需要一次复数乘和两次复数加法运算;
- 经过一次分解后, 共需要复数乘和复数加的次数为 $\frac{N(N+1)}{2}$ 和 $N^2/2$

N=8时第1次分解的运算流图

$$\begin{cases} x_1(n) = x(n) + x(n+N/2) \\ x_2(n) = [x(n) - x(n+N/2)] W_N^n \end{cases}$$

$$\begin{cases} X(2r) = \sum_{n=0}^{N/2-1} x_1(n) W_{N/2}^r \\ X(2r+1) = \sum_{n=0}^{N/2-1} x_2(n) W_{N/2}^r \end{cases}$$



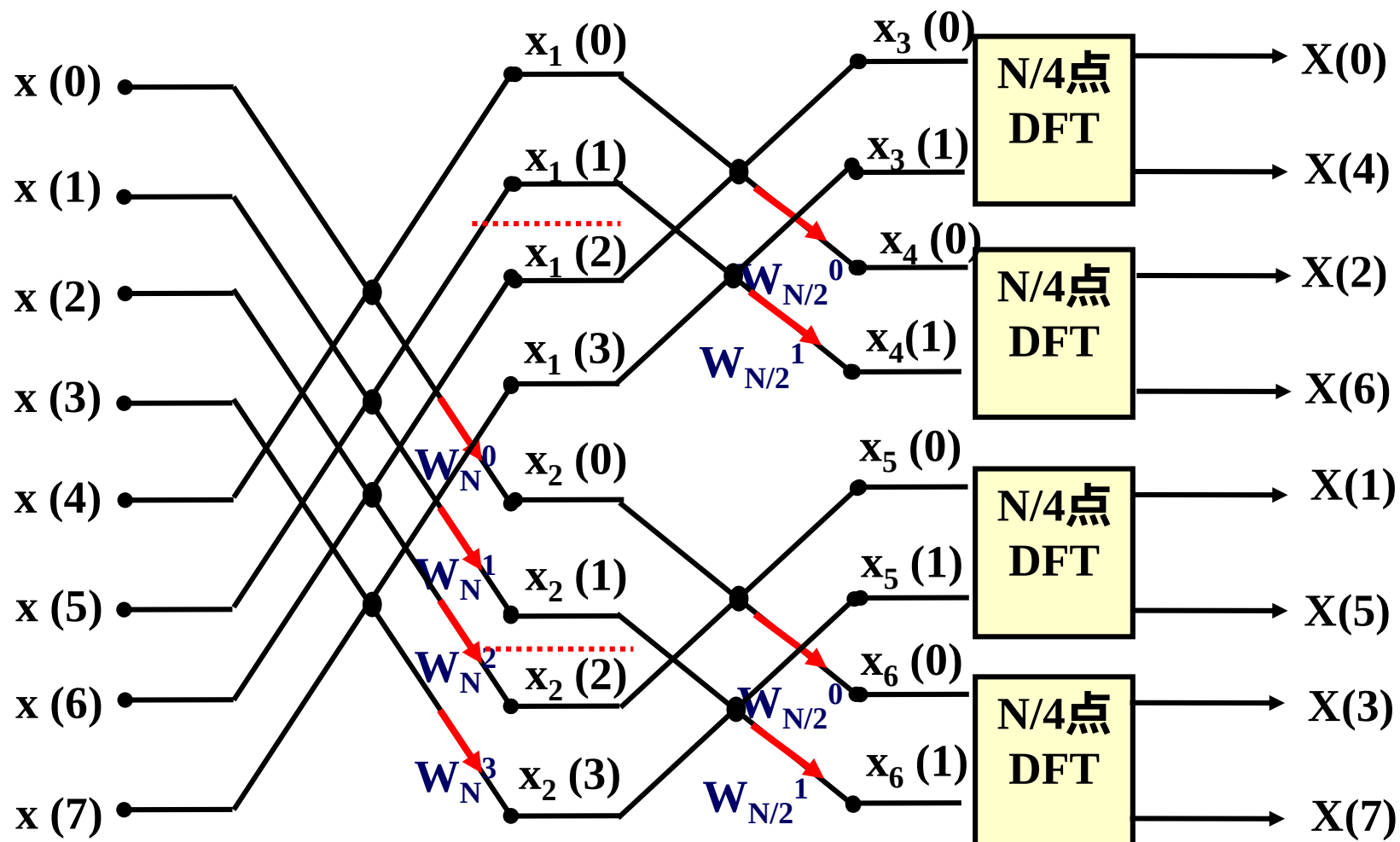
$N=2^M$, $N/2$ 仍是偶数, 继续将 **$N/2$ 点进行分解**。将输入序列 $x_1(n)$ 、 $x_2(n)$ 分别按前、后对半分, 构造出4个长度为 $N/4$ 的子序列。

$$\begin{cases} x_3(n) = x_1(n) + x_1(n+N/4) \\ x_4(n) = [x_1(n) - x_1(n+N/4)] W_{N/2}^n \end{cases}$$

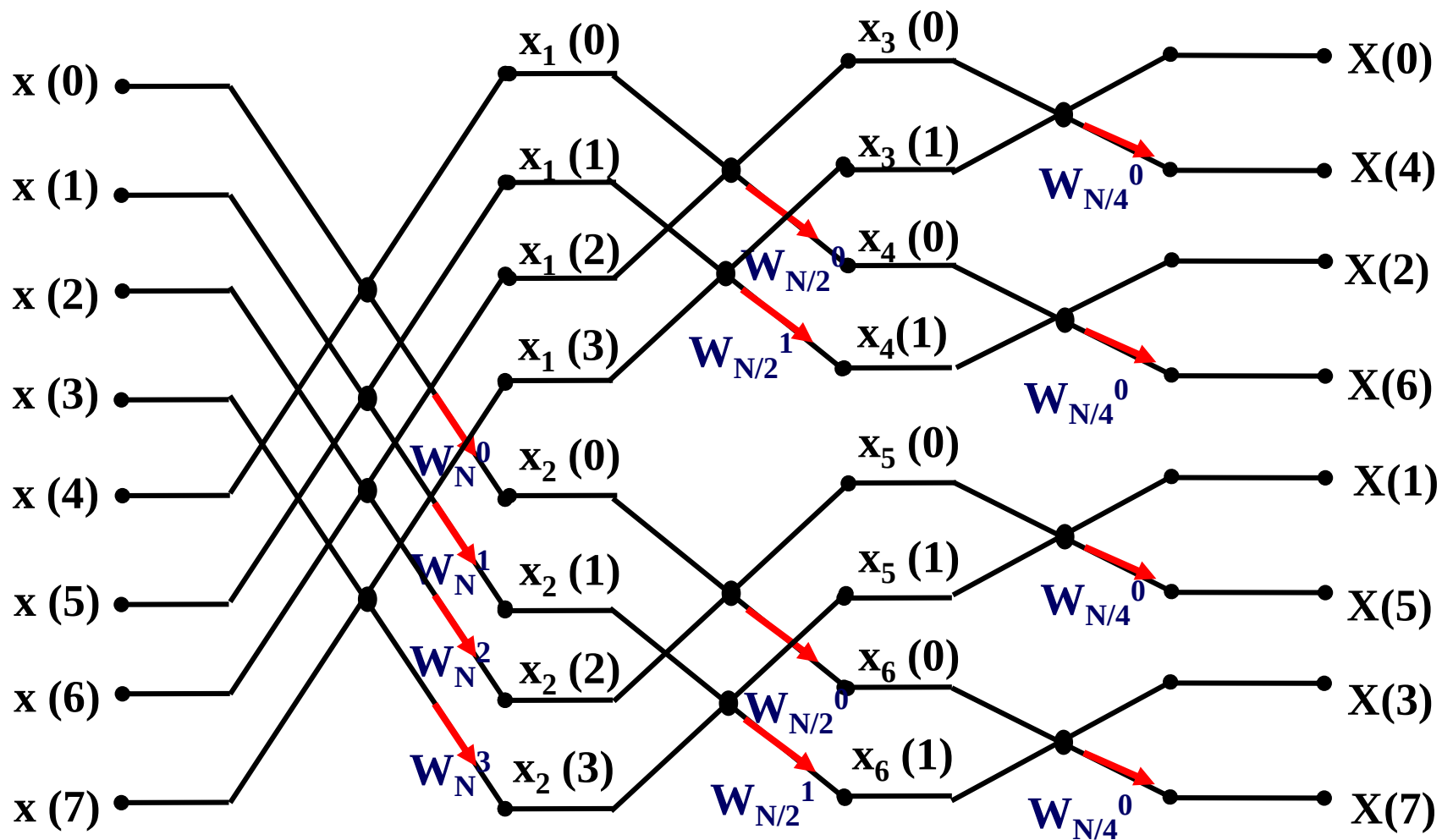
$$n=0, 1, \dots, N/4-1$$

$$\begin{cases} x_5(n) = x_2(n) + x_2(n+N/4) \\ x_6(n) = [x_2(n) - x_2(n+N/4)] W_{N/2}^n \end{cases}$$

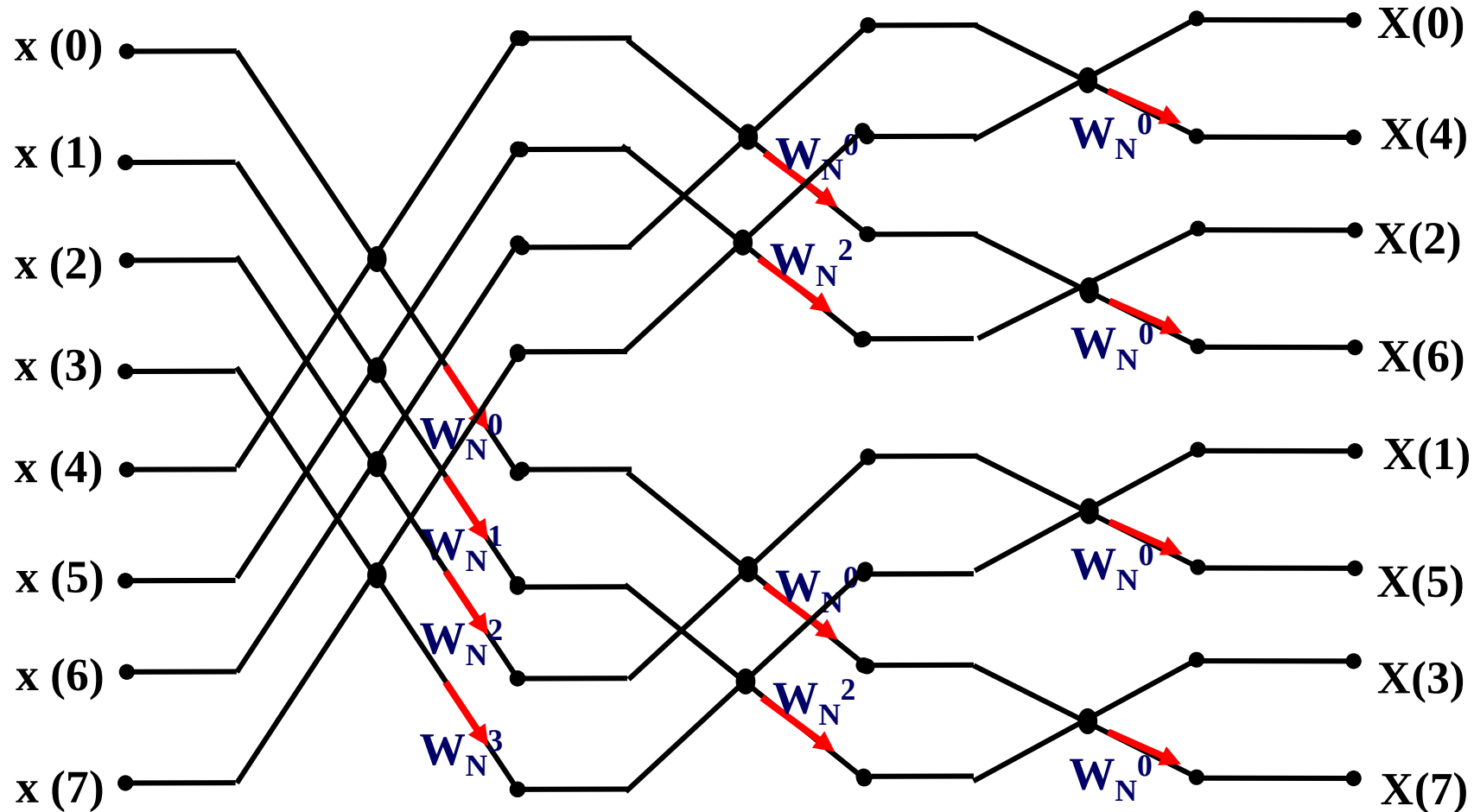
DIF-FFT二次分解运算流图(N=8)



DIF-FFT三次分解运算流图(N=8)



频率抽取法FFT运算流图(N=8)



2、DIF-FFT算法特点

DIF-FFT算法也采用原位运算；

$N=2^M$ 时，共有M级蝶形运算，每级有N/2个蝶形，一个蝶形需要1次复乘，2次复加运算。因此DIT与DIF算法的运算次数相同。

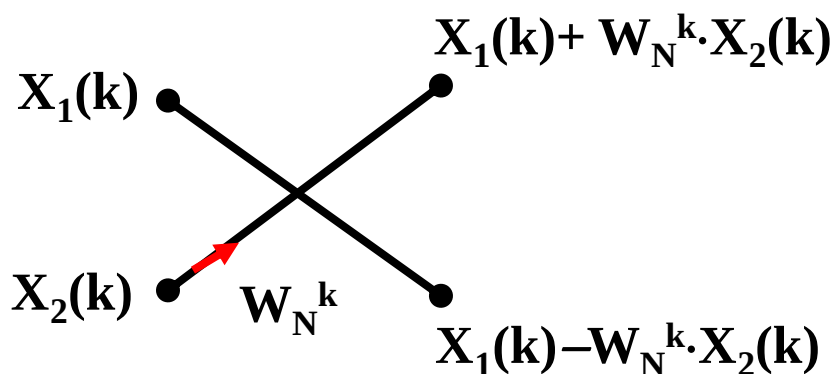
$$\text{复数乘次数: } M \times \frac{N}{2} = \frac{N}{2} \times \log_2 N$$

$$\text{复数加次数: } M \times \frac{N}{2} \times 2 = N \times \log_2 N$$

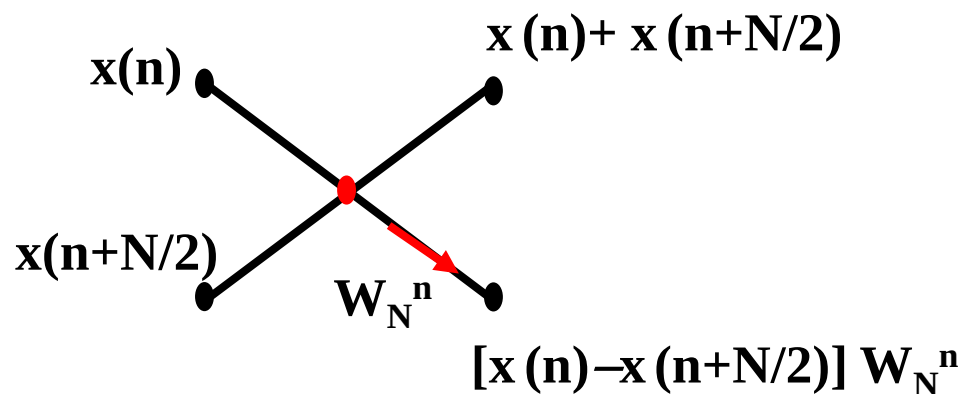
3、DIF-FFT与DIT-FFT算法的不同点

(1) DIF-FFT算法输入序列为顺序，输出为倒序，M级运算后，需对输出数据再进行倒序得到顺序的 $X(k)$ 。而DIT-FFT算法输入为倒序，输出为顺序。

(2) 蝶形运算符号不同：DIT-FFT蝶形是先相乘，后加/减；而DIF-FFT蝶形是先加/减，后相乘。



DIT-FFT蝶形运算流图符号



DIF-FFT蝶形运算流图符号

4.4 IDFT的快速计算方法

1、DFT和IDFT的公式比较

$$X(k) = DFT[x(n)] = \sum_{n=0}^{N-1} x(n)W_N^{kn}$$

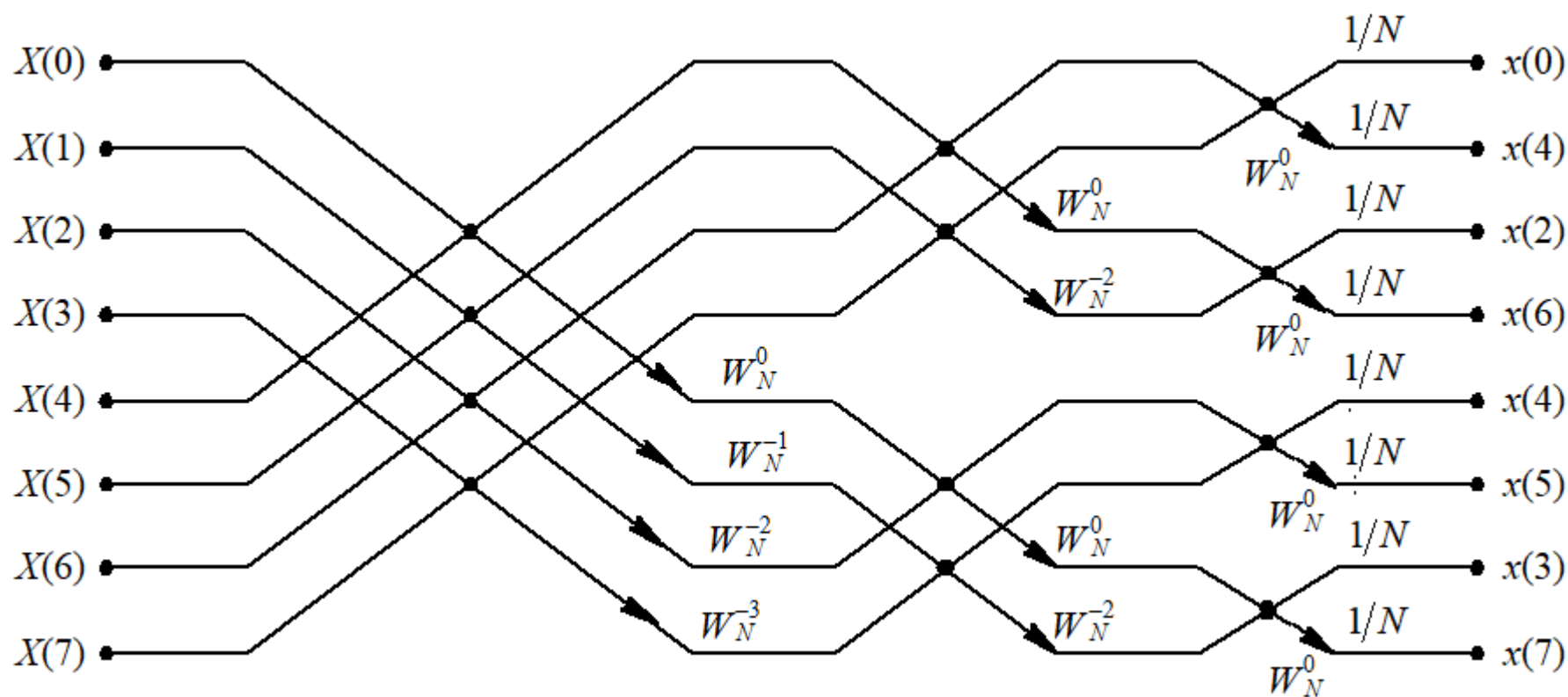
$$x(n) = IDFT[X(k)] = \frac{1}{N} \sum_{k=0}^{N-1} X(k)W_N^{-kn}$$

由公式可以看出：只需将DFT的旋转因子 W_N^{kn} 变为 W_N^{-kn} ，最后结果乘以

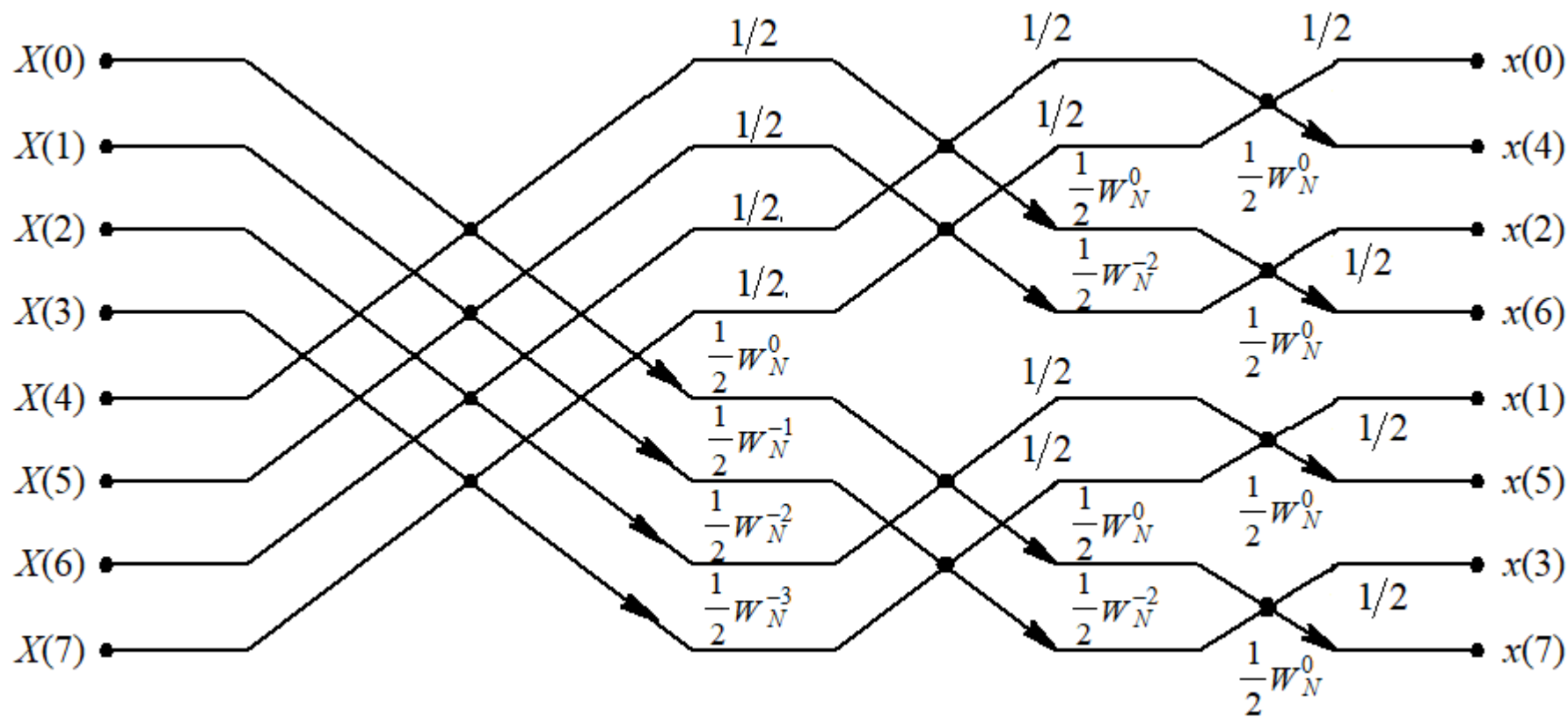
$1/N$ ，就是IDFT的运算公式。

2、用FFT运算流图实现IDFT快速算法（方法1）

- 将DIT-FFT或DIF-FFT蝶形运算流图中旋转因子 W_N^p 改为 W_N^{-p}
- 在IDFT快速算法的最后结果输出前，乘以 $1/N$ 常数；或在每一级运算中，输出支路分别乘以 $1/2$ ，实现系数分级分担。
- 在IDFT快速算法中，输入序列为 $X(k)$ ，而输出序列为 $x(n)$ 。



IFFT运算流图



IFFT运算流图

3、直接调用FFT程序（或函数）实现IFFT的计算（方法2）

对FFT实现流程做一些修改，调用FFT程序实现IFFT的快速计算。

$$\begin{aligned} x(n) = IDFT[X(k)] &= \frac{1}{N} \sum_{k=0}^{N-1} X(k) W_N^{-kn} \\ &= \frac{1}{N} \left[\sum_{k=0}^{N-1} X^*(k) W_N^{kn} \right]^* = \frac{1}{N} \{ DFT[X^*(k)] \}^* \end{aligned}$$

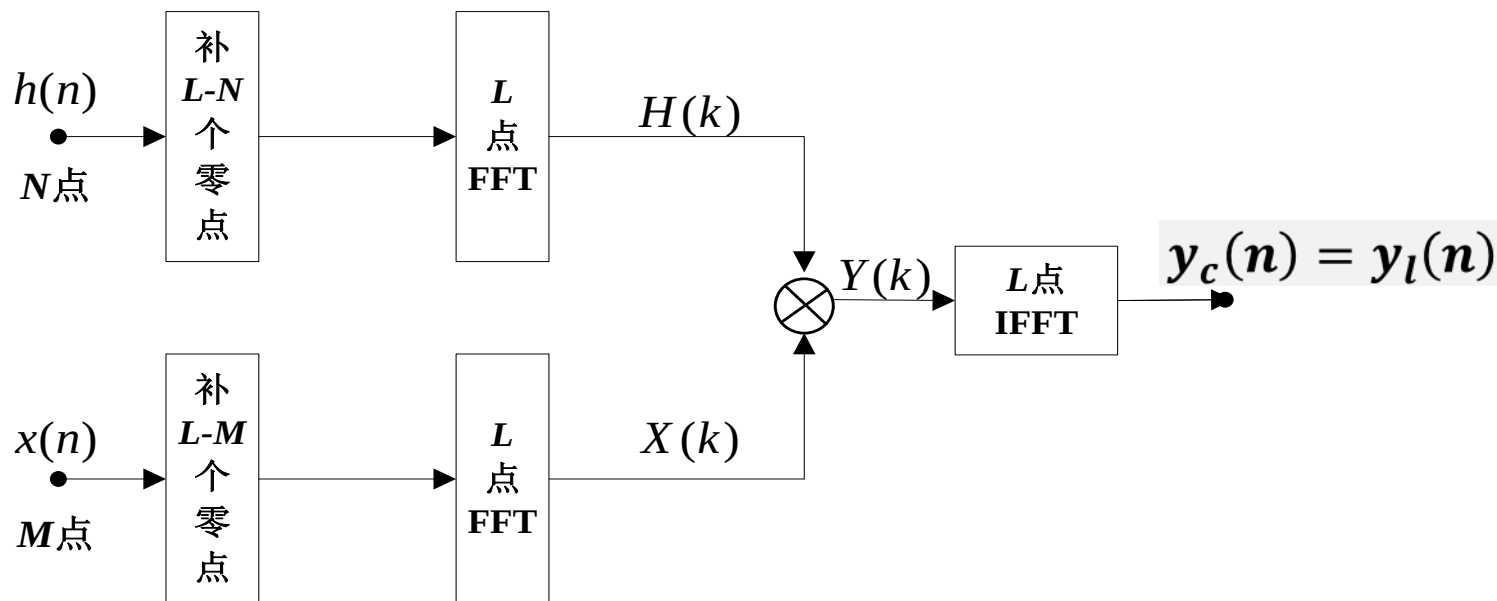
实现步骤：

- 先将 $X(k)$ 取**共轭**，得到 $X^*(k)$ ；
- 调用FFT子程序计算出 $DFT[X^*(k)]$ 的值；
- 对输出序列取**共轭**，并乘以 $1/N$ 常数。

虽用了2次取共轭运算，但可和FFT共用一个子程序，实现方便。

(1) 再次理解快速卷积法：用FFT实现线性卷积

某LTI系统，输入序列 $x(n)$ 的长度为 M ，系统的 $h(n)$ 的长度为 N ，求系统输出



注意 L 取值的要求： (1) $L \geq M+N-1$; (2) $L=2^J$

例3：一个长100点，另一个长25点的复序列进行线性卷积。如果借助于基2FFT进行快速卷积，要得到与线性卷积同样结果，所需做的FFT次数是_____，总的复数乘法计算量（不考虑旋转因子和序列的特殊值）是_____。

3次

$$L \geq M+N-1=124 \quad ; \quad L=2^7=128$$

$$3 \times N/2 \times \log_2 N + N = 3/2 \times 128 \times 7 + 128 = 1472 \text{次}$$

(2) 实序列的FFT算法

例题4：通过计算一个 N ($N=2^M$) 点FFT (DFT)，得到两个不同实序列 $x_1(n)$ 和 $x_2(n)$ 的 N 点FFT (DFT)。(教材习题的第3题)

解：构成新序列 $x(n)=x_1(n)+jx_2(n)$

对 $x(n)$ 进行 N 点的FFT得到 $X(k)=\text{DFT}[x(n)]$

$$\text{因为 } X(k) = X_{\text{ep}}(k) + X_{\text{op}}(k)$$

$$X_{\text{ep}}(k)=\text{DFT}[x_1(n)]=1/2[X(k)+X^*(N-k)]$$

$$X_{\text{op}}(k)=\text{DFT}[jx_2(n)]=1/2[X(k)-X^*(N-k)]$$

根据对称性，得到：

$$X_1(k)=\text{DFT}[x_1(n)]=1/2[X(k)+X^*(N-k)]$$

$$X_2(k)=\text{DFT}[x_2(n)]=-j1/2[X(k)-X^*(N-k)]$$

例题5： $x(n)$ 为 N 点实序列， $N=2^M$ ，设计一种算法，只计算 $N/2$ 点DFT(FFT)，得到 N 点的 $X(k)$ 。(教材习题的第4题 (1))

步骤1： 取 $x(n)$ 偶数序号点和奇数序号点分别作为新序列 $y(n)$ 的实部和虚部，即：

$$\begin{aligned} x_1(n) &= x(2n) \\ x_2(n) &= x(2n+1) \end{aligned} \quad \Rightarrow \quad y(n) = x_1(n) + jx_2(n), \quad n = 0, 1, \dots, \frac{N}{2} - 1$$

步骤2： 对 $y(n)$ 进行 $N/2$ 点FFT，得到 $Y(k)$ 。

$$\begin{aligned} X_1(k) &= \text{DFT}[x_1(n)] = Y_{ep}(k) = \frac{1}{2}[Y(k) + Y^*(\frac{N}{2} - k)] \\ X_2(k) &= \text{DFT}[x_2(n)] = -jY_{op}(k) = -j\frac{1}{2}[Y(k) - Y^*(\frac{N}{2} - k)] \end{aligned} \quad k = 0, 1, \dots, \frac{N}{2} - 1$$

步骤3： 根据DIT-FFT的思想

$$X(k) = X_1(k) + W_N^k X_2(k)$$

$$X(k + \frac{N}{2}) = X_1(k) - W_N^k X_2(k)$$

$$k = 0, 1, \dots, \frac{N}{2} - 1$$

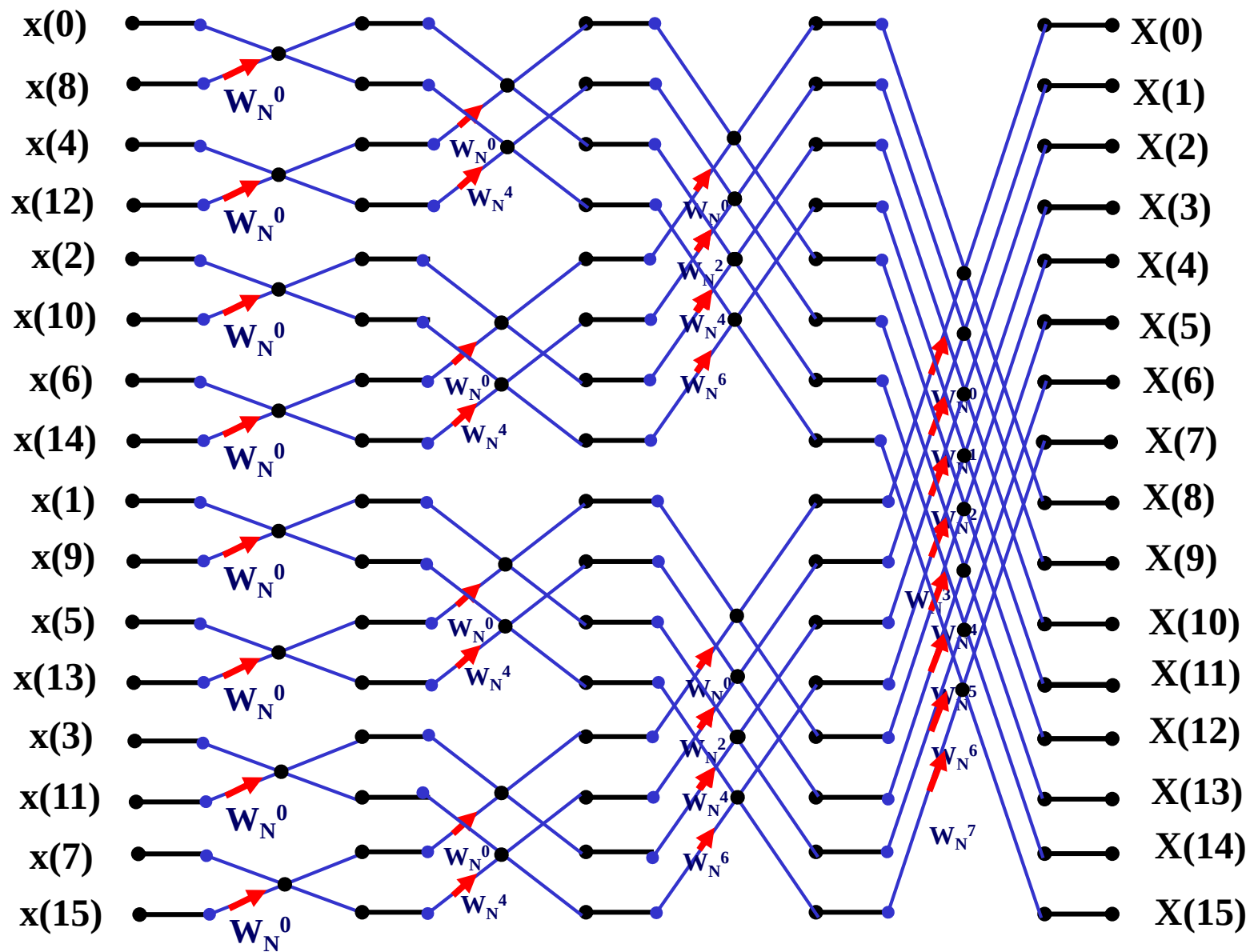
或

$$X(k) = X_1(k) + W_N^k X_2(k)$$

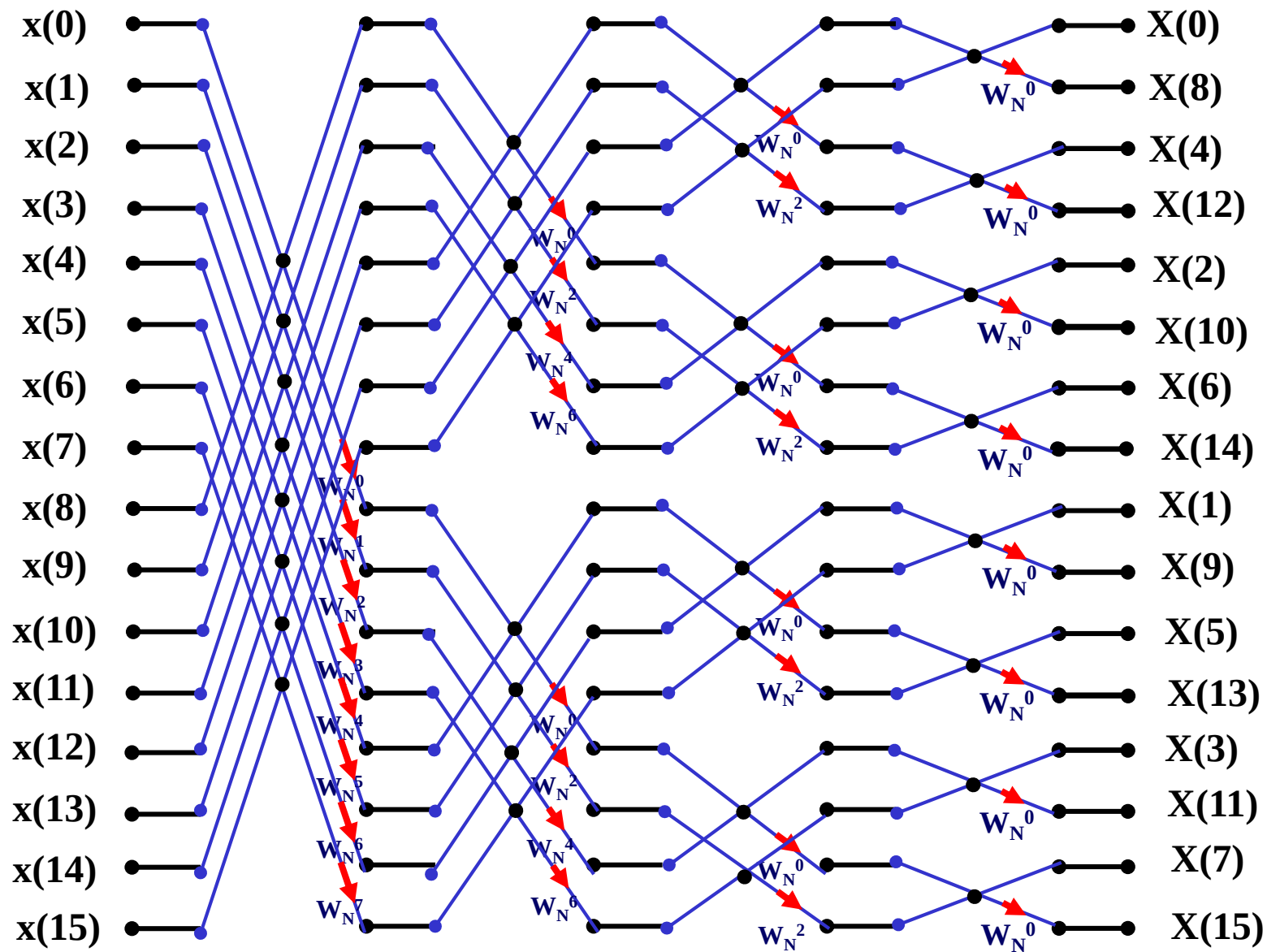
$$X(\frac{N}{2}) = X_1(0) - X_2(0)$$

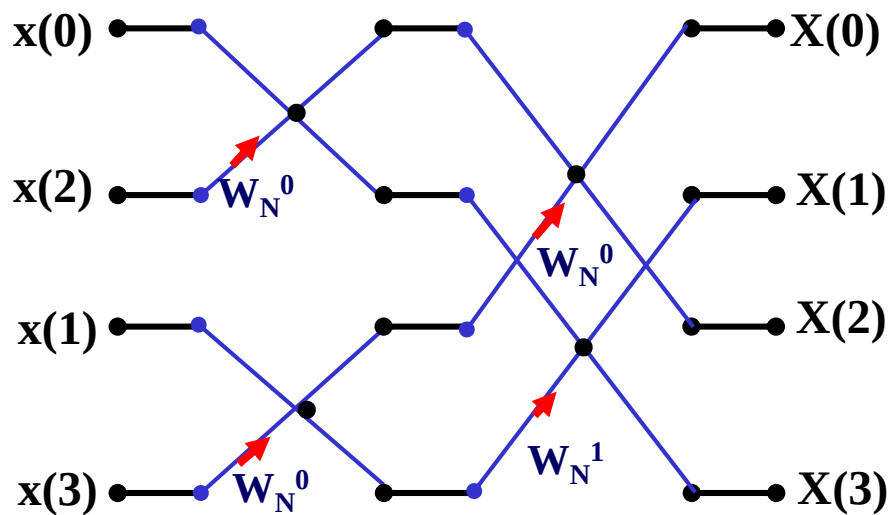
$$X(N - k) = X^*(k)$$

N=16点DIT-FFT运算流图

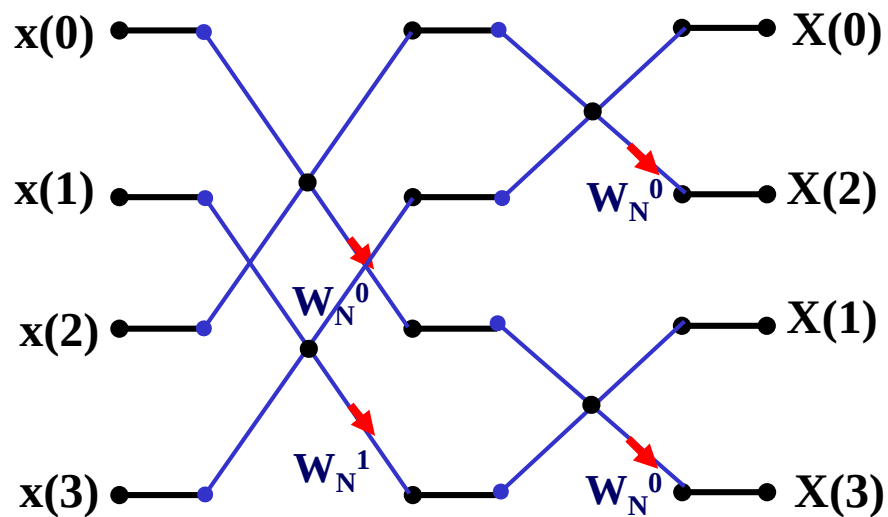


N=16点DIF-FFT运算流图





N=4点DIT-FFT运算流图



N=4点DIF-FFT运算流图

第4章 作业

- 1、利用FFT处理器对连续时间信号进行谱分析，要求指标：① 频谱分辨率为 $F \leq 5\text{Hz}$ ；② 信号的最高频率 $f_{\max} \leq 1.25\text{kHz}$ ；求：(1) 信号最小记录时间 T_p ；(2) 最大的采样间隔 T_s ；(3) 最小的采样点数 $N=?$
- 2、画出4点的DIT-基2FFT的运算流图，并利用流图计算序列 $x(n)=\{1,2,-4,2\}$ 的 4点DFT。画出4点的DIF-基2FFT的运算流图，并利用流图计算序列 $x(n)=\{1,2,-1,-2\}$ 的 4点DFT。
- 3、教材，第1题，注意：把题中的N改为64。